Summer Research Internship 2026

Submitted by

Harsh Raj

B.Tech 3rd Year, Computer Science and Engineering

Indian Institute of Information Technology, Kalyani

Internship Organization

Internship carried out at

Jawaharlal Nehru Centre for Advanced Scientific Research

Under the guidance of

Prof. K. S. Narayan Sir

Automated Multi-Panel Image Processing Pipeline

Fisheye Correction, Adaptive Grid Detection, and Cell-Wise Image Stitching

1. Introduction

This project implements an automated image-processing system that takes a set of sixteen photographs of a grid-based panel — captured through a wide-angle (fisheye) camera lens — and converts them into clean, perfectly aligned grids of individual cell images. The system is composed of two parts: a processing engine that performs all the image-analysis work, and a desktop application that lets a user run the engine, watch its progress, and inspect the results without writing any code.

The motivation behind the system is that raw photographs of a multi-cell panel are rarely usable directly. Lens distortion bends straight lines, ambient lighting is uneven across the frame, and the exact position of each cell boundary differs slightly from photo to photo. The pipeline therefore applies a sequence of optical and image-processing corrections so that, regardless of how the original photograph looked, the final output is a consistently cropped, evenly lit, and precisely segmented set of cell images.

Each input photograph may contain a panel arranged as anywhere from a single cell (1×1) up to a sixteen-cell grid (4×4). After all sixteen photographs in a batch have been processed, the system also produces one final “bird's-eye” composite image that places every processed panel side by side, giving a single overview of the entire batch.

2. System Architecture

The system follows a simple two-layer design. The Graphical User Interface (GUI) layer is responsible only for user interaction — selecting an input folder, choosing the grid size, displaying progress, and showing results. All of the actual image analysis is delegated to a separate Processing API layer, which can equally well be called from the GUI, from a command line, or from any other Python program. This separation means the processing logic can be reused, tested, or automated independently of the interface that triggers it.

Figure 2. System Architecture

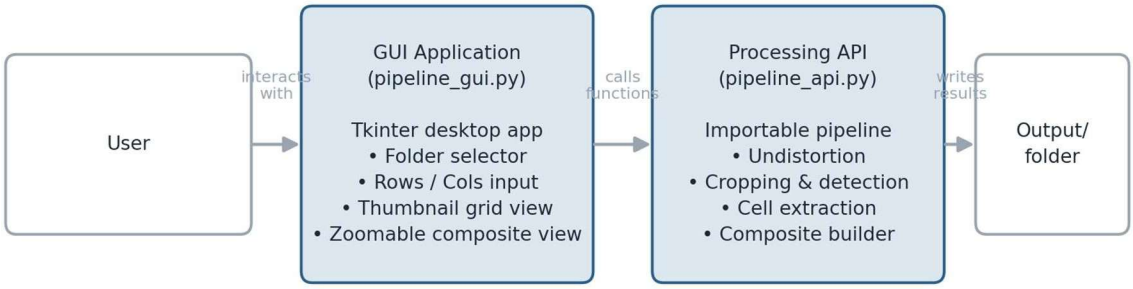


Figure 2. The GUI and the processing engine communicate through a single function call; the engine reads images from the input folder and writes all results to a structured output folder.

When the user presses “Run,” the GUI hands the folder path and the chosen grid size to the Processing API, which works through all sixteen images and returns the locations of every output file once finished. The GUI then loads those files back in for display, so the two layers never need to share anything more than file paths and simple result data.

3. The Image Processing Pipeline

The core of the project is a nine-stage pipeline applied independently to each of the sixteen input photographs. The stages run in strict sequence, with each stage's output feeding directly into the next, as summarised below.

Figure 1. Image-Processing Pipeline — Stage Overview

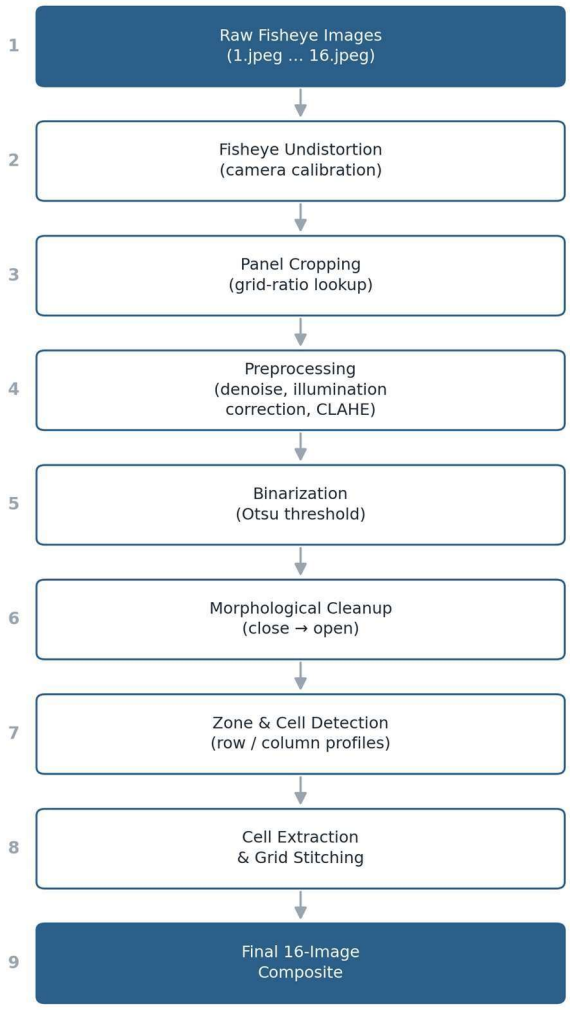


Figure 1. The nine sequential stages applied to every input photograph.

3.1 Image Acquisition

Each batch consists of sixteen photographs of the same physical panel, captured one after another, typically representing sixteen different samples, positions, or time points being documented under identical conditions. The system expects them to be named sequentially so that they can be matched and processed automatically.

3.2 Fisheye Lens Undistortion

Wide-angle camera lenses bend straight lines near the edges of the frame, a visual artefact known as barrel or fisheye distortion. Because the camera's internal optical properties (its focal length, optical centre, and lens-distortion coefficients) are known in advance from a one-time calibration, the system can mathematically reverse this bending. A correction map is computed once for the image resolution in use and then reused for every photograph, so the optical correction itself is fast even though it is applied sixteen times per batch. A tunable balance setting controls how much of the original wide field of view is preserved versus how much black border is trimmed away during the correction.

3.3 Panel Localization and Cropping

Once distortion is removed, the system isolates just the panel area and discards the surrounding background, mounting hardware, or frame edges. Because the exact position of the panel within the frame depends on how many rows and columns of cells it contains, the system keeps a lookup table of crop boundaries — one entry for every supported grid size from 1×1 up to 4×4 — each tuned so that the cropped region contains the full panel with minimal extra background. If the panel itself carries printed text indicating its row and column count, the system can optionally read that text automatically (via optical character recognition) instead of relying on a manually specified grid size.

3.4 Image Preprocessing

Before any decisions are made about where cell boundaries lie, the cropped image is cleaned up in three steps:

- Denoising — a light blur removes small-scale sensor noise without blurring genuine structure.
- Illumination correction — an estimate of the smooth background lighting is computed and divided out, so that uneven lighting across the panel (brighter in the centre, dimmer at the edges, for example) no longer affects later steps.
- Local contrast enhancement — adaptive histogram equalisation boosts faint detail within small regions of the image without over-amplifying noise, making true cell boundaries easier to detect even on a low-contrast photograph.

3.5 Binarization

The enhanced grayscale image is then converted into a strict black-and-white image. The system uses Otsu's method, a well-established statistical technique that automatically finds the brightness threshold that best separates two populations of pixels — in this case, panel content versus background — without requiring a fixed threshold to be set by hand. This makes the system robust to photographs taken under slightly different lighting conditions.

3.6 Morphological Refinement

Binarization alone often leaves small gaps inside cell regions or stray isolated specks of noise. A pair of standard morphological operations is applied to clean this up: a “closing” operation first fills in small gaps, and a subsequent “opening” operation then removes any remaining isolated noise pixels. The result is a clean black-and-white mask whose solid regions correspond to the genuine structure of the panel.

3.7 Zone and Cell Boundary Detection

This is the most analytically involved stage. The system computes a brightness profile across the image — essentially, an average darkness value for each row and, separately, for each column — and looks for distinct “valleys” where the profile drops well below its surrounding local peak. These valleys correspond to the thin gaps or separators that exist between adjacent cells on the physical panel. Row boundaries are located first, using a central horizontal band of the image to avoid being confused by anything near the very top or bottom edge; column boundaries are then located using the same approach within the vertical span the rows define. If more candidate boundaries are found than the expected grid size calls for, the weakest (narrowest) ones are discarded; if too few are found, the stage reports that the grid could not be reliably located.

3.8 Cell Extraction and Edge Padding

With every cell's exact boundary now known, the system crops each cell directly from the original (non-binarized) photograph, preserving full image detail and colour. A small uniform padding is added around each cell so that thin borders or partially-cut content near a cell's edge are not lost. Additional padding is applied specifically at the outer edges of the panel whenever the profile analysis detects an unusually wide dark margin there, which typically indicates that a sliver of an adjoining cell or border was nearly cut off.

3.9 Grid Stitching

All of the individual cell crops belonging to one photograph are then reassembled into a single image, arranged in their original row-and-column order with a thin, even margin between cells. This produces one clean “stitched” grid image per input photograph — effectively a tidied-up, perfectly aligned version of the original panel photo.

3.10 Final Composite Construction

After all sixteen photographs have been processed independently (and, as described in Section 4, this happens in parallel rather than one at a time), their sixteen stitched grid images are combined into a single overview image. Each stitched image is first normalised onto a common brightness scale, so that lighting differences between photographs do not make the final overview look inconsistent. The sixteen images are then arranged into two side-by-side columns

of eight rows each, with a slightly larger gap inserted between the fourth and fifth row of each column to visually separate the batch into two natural groups of four. The result is a single image that lets a user review the outcome of an entire sixteen-photograph batch at a glance.

Figure 5. Final Composite Layout
(2 columns × 8 rows, segmented in groups of 4)

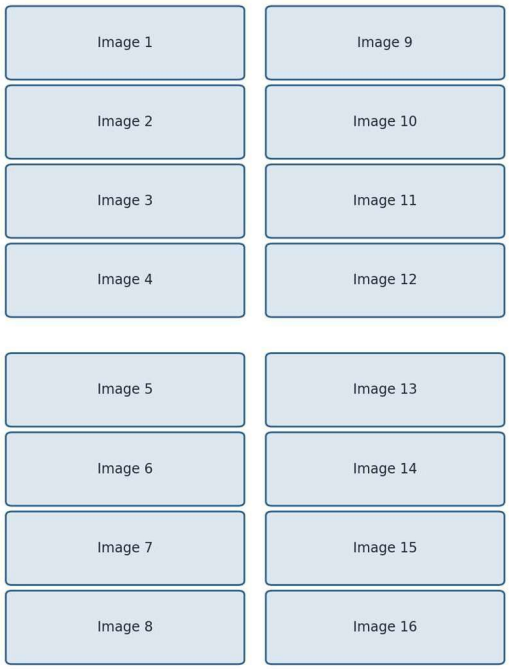


Figure 5. Layout of the final composite image: sixteen per-photograph results are tiled into two columns of eight, with a visual gap separating each group of four.

4. Parallel Processing and Performance

Because the nine-stage pipeline described above is applied independently and identically to each of the sixteen input photographs, there is no need to process them one after another. The system instead processes multiple photographs at the same time using a pool of worker threads, with the number of simultaneous workers chosen automatically based on how many processor cores are available on the machine (up to a maximum of sixteen, one per photograph). Since the heavy lifting in each stage is handled by an optimised image-processing library that runs outside of Python's own execution lock, the workers genuinely run in parallel rather than merely taking turns, so total processing time for a full batch is reduced roughly in proportion to the number of available cores rather than scaling linearly with sixteen separate, sequential runs.

5. Supported Panel Grid Configurations

The system is not limited to a single panel layout. It supports any combination of one to four rows and one to four columns — sixteen distinct configurations in total — each with its own pre-tuned cropping boundaries, as described in Section 3.3. The number of individual cells extracted from a single photograph therefore ranges from just one (a 1×1 panel) up to sixteen (a 4×4 panel), as shown below. The system's default configuration is a 3×3 grid, producing nine cells per photograph.

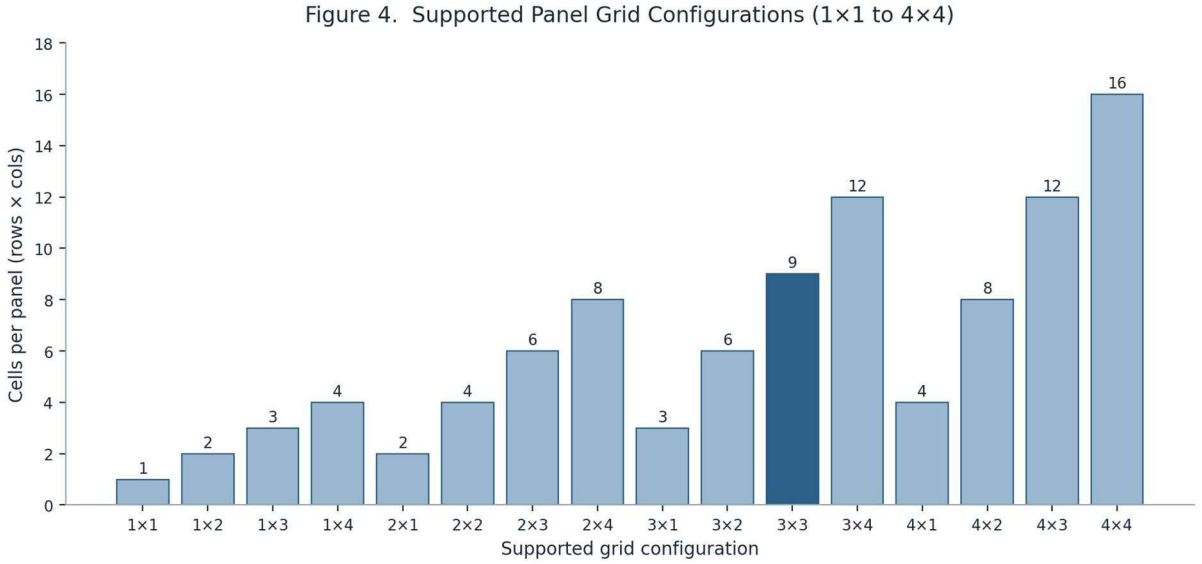


Figure 4. Every supported grid layout and the resulting number of cells extracted from each panel. The 3×3 default configuration is highlighted.

6. Output Data Organization

Results are written to a structured output folder that mirrors the input batch. For every photograph, a numbered subfolder holds that photograph's stitched grid image along with every individual cell crop extracted from it; once all sixteen photographs have been processed, a single final composite image is placed at the top level of the batch's output folder. For the system's default 3×3 configuration, a full batch run therefore produces 144 individual cell images, 16 per-photograph stitched grids, and 1 overall composite — 161 files in total, as illustrated below.

Figure 3. Output File Composition per Run
(default 3×3 grid, 16 input images)

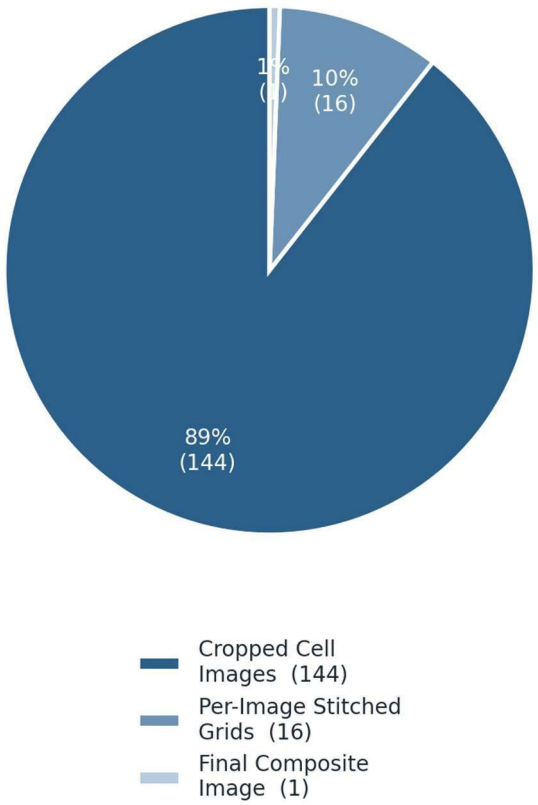


Figure 3. Composition of the output files generated by a single batch run, assuming the default 3×3 grid configuration.

7. Graphical User Interface

The desktop application provides a straightforward way to run the pipeline and review its results without needing to work with the underlying code directly. Its main features are:

- Folder selection — a standard file-browser dialog lets the user point the application at the folder containing the sixteen input photographs.
- Grid configuration — simple numeric controls let the user specify how many rows and columns the panel contains before processing begins.
- Background processing — the pipeline runs on a separate background thread, so the interface remains responsive and a progress indicator and status message keep the user informed while processing is underway.
- Per-photograph review (Tab 1) — a four-by-four grid of thumbnails fills in live as each of the sixteen photographs finishes processing, with a clear visual marker on any photograph that failed to process.
- Composite review (Tab 2) — the final overview image is displayed in a custom viewer that supports smooth panning and zooming, including two-finger trackpad gestures, click-and-drag panning, dedicated zoom buttons, and a horizontal mirror toggle. While the user is actively zooming, the viewer uses a fast, lower-quality resampling method to stay responsive, automatically switching to a sharper, higher-quality method a fraction of a second after the gesture settles.
- Saving — the finished composite image can be exported to any location the user chooses, independent of the structured output folder the pipeline writes by default.

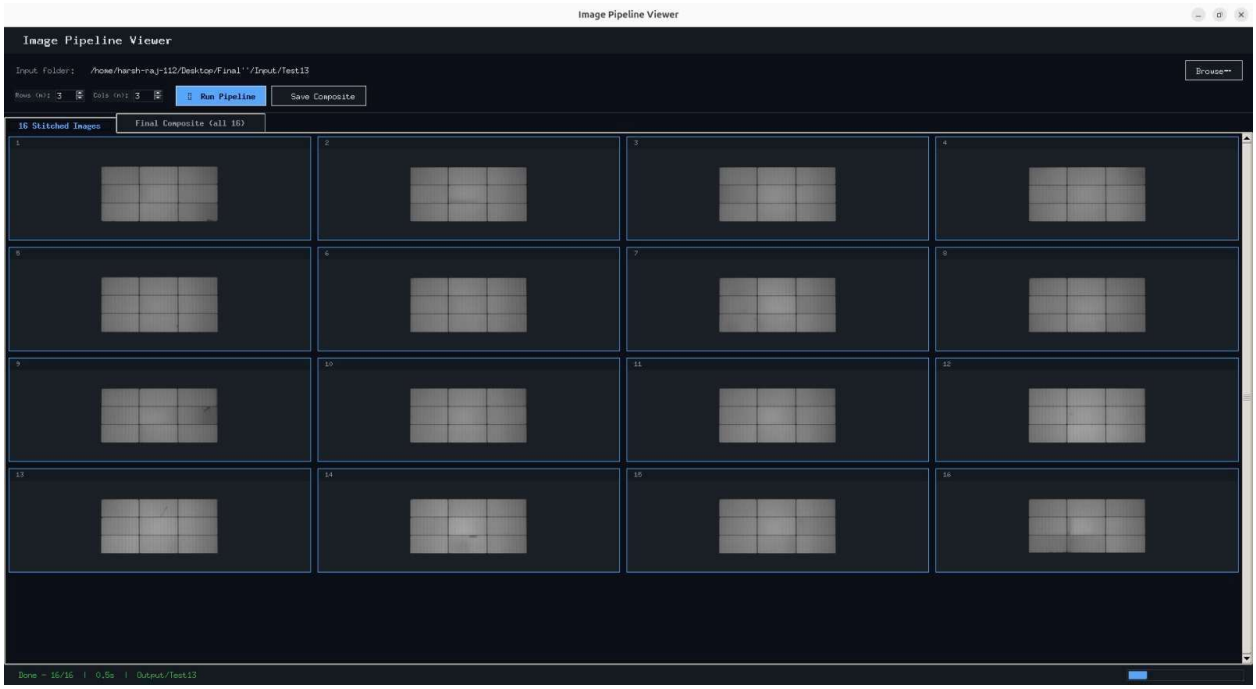
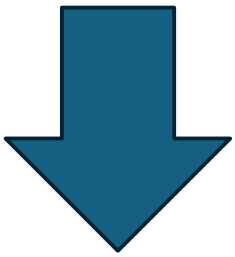


Image Pipeline Viewer
Image Pipeline Viewer
Input Folder: /home/harsh-raj-112/Desktop/Final''/Input/Test13
Browse...
Rows (H): 3
Cols (W): 3
Run Pipeline
Save Composite
16 Stitched Images
Final Composite (all 16)
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
Done — 16/16 | 0.5s | Output/Test13



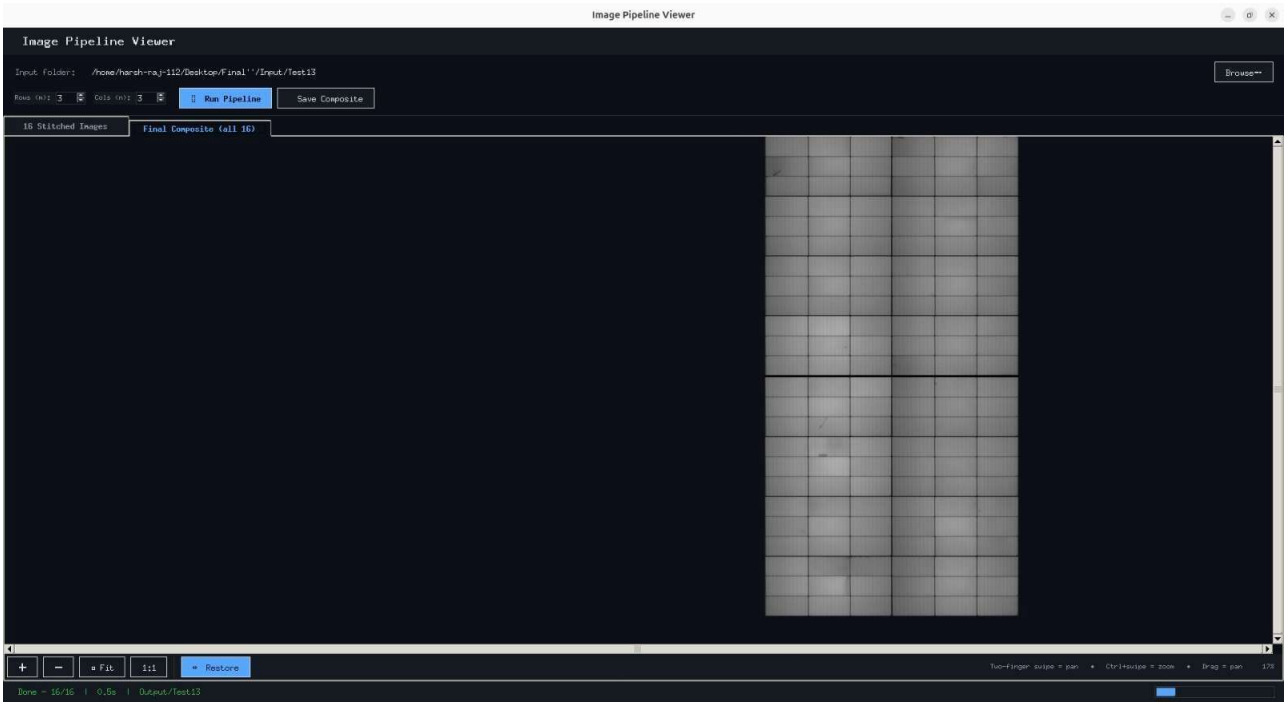


Image Pipeline Viewer
Image Pipeline Viewer
Input Folder: /home/harsh-raj-112/Desktop/Final''/Input/Test13
Browse...
Rows (H): 3
Cols (W): 3
Run Pipeline
Save Composite
16 Stitched Images
Final Composite (all 16)
+
−
Fit
1:1
Restore
Two-finger swipe = pan • Ctrl+swipe = zoom • Drag = pan
Done — 16/16 | 0.5s | Output/Test13

8. Conclusion

This project demonstrates a complete, self-contained pipeline for turning raw, distorted photographs of a multi-cell panel into clean, precisely segmented, and consistently presented image data. By combining a one-time optical calibration, adaptive image enhancement, statistically grounded thresholding, and a profile-based boundary-detection algorithm, the system is able to automatically locate and extract individual cells from panels of varying sizes without manual intervention for each photograph. Wrapping this processing engine in a responsive desktop interface, and running the sixteen photographs of a batch in parallel, makes the overall system both practical to operate and efficient to run at the batch sizes it was designed for.

Multi-Segment Optical Image Stitching Pipeline

Border Removal, Strip Extraction, Geometric Correction, and Panoramic Stitching

1. Introduction

This project implements an automated pipeline that takes three raw photographs — referred to as segments — produced by an optical imaging instrument, and combines them into a single clean, panoramic composite image. Photographs of this kind are common when an instrument's capture mechanism records a wide scene as several narrower, overlapping or adjacent frames rather than one continuous image; each frame typically carries its own white margin, slight misalignment, and uneven framing that must be corrected before the frames can be joined together convincingly.

The pipeline corrects each of the three segments independently — removing border artefacts, isolating the region that actually contains useful image content, correcting its geometry, and trimming instrument-specific offsets — before stitching all three into one wide image. Two versions of the final composite are produced: a normal left-to-right arrangement, and a fully mirrored arrangement that is useful when the scan needs to be reviewed or compared in the opposite orientation.

Because the same sequence of corrections is applied to each of the three segments independently, the system processes all three at the same time using parallel execution, rather than working through them one at a time.

2. System Architecture

The system is a single self-contained script rather than a separate engine-and-interface pair: it locates the three expected segment files in an input folder, processes each one through an identical correction pipeline, and then stitches the results into the two output composites described above. A lightweight timing utility records how long each major phase takes and prints a simple text-based report once the run completes.

Figure 2. System Architecture

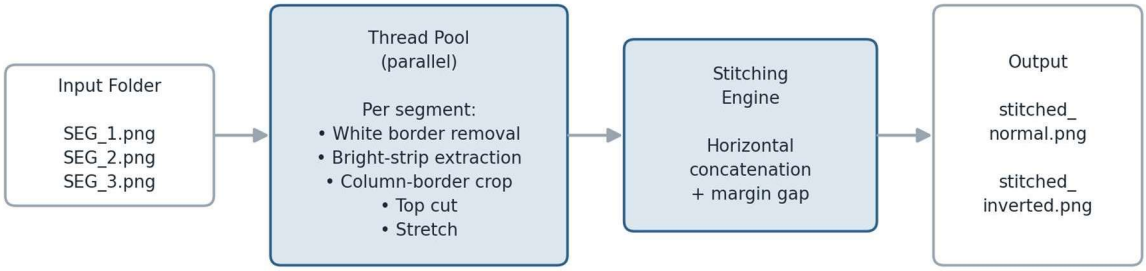


Figure 2. The three input segments are corrected in parallel and then combined by a single stitching stage into two output composites.

3. The Per-Segment Processing Pipeline

Each of the three segments passes through the same five-stage correction sequence before stitching. The stages run in a fixed order, since each one depends on the cleaned-up output of the stage before it.

Figure 1. Per-Segment Processing Pipeline

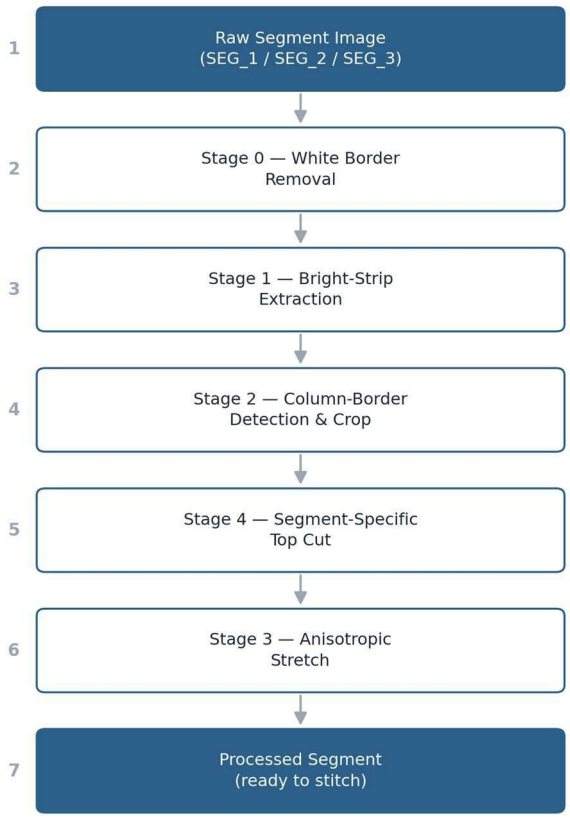


Figure 1. The five correction stages applied to every segment before it is stitched into the final composite.

3.1 Stage 0 — White Border Removal

Photographs exported by imaging instruments frequently include a flat white or near-white margin around the actual content, left over from the instrument's display or printing layout. The system examines each row and each column of the photograph and classifies it as “white” if a large proportion of its pixels are brighter than a fixed threshold. Starting from each of the four edges, it then walks inward until it reaches the first row or column that is no longer predominantly white, and crops away everything outside that boundary. A small safety check prevents the crop from removing so much that almost no content would remain, in case a segment happens to be unusually bright overall.

3.2 Stage 1 — Bright-Strip Extraction

With the white margin removed, the system looks across the width of the image for the region that actually contains the instrument's captured data, which appears noticeably brighter than the surrounding background once the image is reduced to a column-by-column brightness profile. The profile is smoothed to reduce noise, and the widest two stretches of above-threshold brightness are treated as the candidate data region. The exact left and right edges of that region are then refined by walking outward from each candidate boundary until the smoothed brightness profile stops decreasing — that is, until the nearest local valley is reached — with a small additional allowance added on the right-hand side to avoid clipping content.

3.3 Stage 2 — Column-Border Detection and Crop

The extracted strip can still contain a thin instrument-drawn border or frame around the data itself. To find it, the system again builds a column brightness profile, smooths it with a Gaussian filter, and computes the rate of change (gradient) of that profile from one column to the next. Sharp jumps in this gradient — detected as statistically significant peaks — mark likely border edges. For each such peak, the system looks in a small neighbourhood for the exact darkest point, treating that as the true edge position, since borders are typically a thin dark line rather than a single pixel. The leftmost and rightmost edge positions found this way define the final crop, isolating just the instrument's actual data region.

3.4 Stage 4 — Segment-Specific Top Cut

Because the three segments are not always framed identically by the capture instrument, a small, fixed number of pixels is trimmed from the top of each segment individually, with a different amount configured for each of the three segment positions. This compensates for consistent, segment-specific vertical offsets that the earlier, content-based stages are not designed to detect, since those stages reason about brightness rather than fixed instrument geometry.

3.5 Stage 3 — Anisotropic Stretch

Finally, each cropped segment is resized using independent horizontal and vertical scale factors. Unlike an ordinary resize, this anisotropic stretch can correct for a known proportional distortion introduced by the imaging instrument's optics or scan geometry — for example, compressing or stretching the image more in one direction than the other — so that the corrected segment reflects the true proportions of the captured scene before it is joined to its neighbours.

4. Parallel Processing

Since Stages 0 through 4 are applied identically and independently to each of the three segments, the system processes all three at once using a small pool of worker threads — one worker per segment — instead of working through them sequentially. Because the underlying image-processing operations run outside of Python's own execution lock, the three segments are genuinely corrected in parallel, which keeps total processing time close to the time needed for the single slowest segment rather than the sum of all three.

5. Stitching and Output Variants

Once all three segments have been corrected, they are joined left-to-right into a single wide composite image, with a thin, consistent gap inserted between neighbouring segments and any height difference between segments padded out so that every segment lines up along a common top edge. This produces the normal composite, stitched_normal.png.

A second, inverted composite is then produced for cases where the opposite orientation is needed for review or comparison. Each corrected segment is individually mirrored left-to-right, and the order of the first and last segment is also swapped before stitching. Mirroring every segment and reversing their order has the same overall effect as mirroring the entire normal composite as one image — so the inverted output is, in effect, a full horizontal mirror of the normal result, produced without having to re-run the correction pipeline a second time.

Figure 5. Normal vs. Inverted Stitch Arrangement
(inverted = full horizontal mirror of the normal composite)

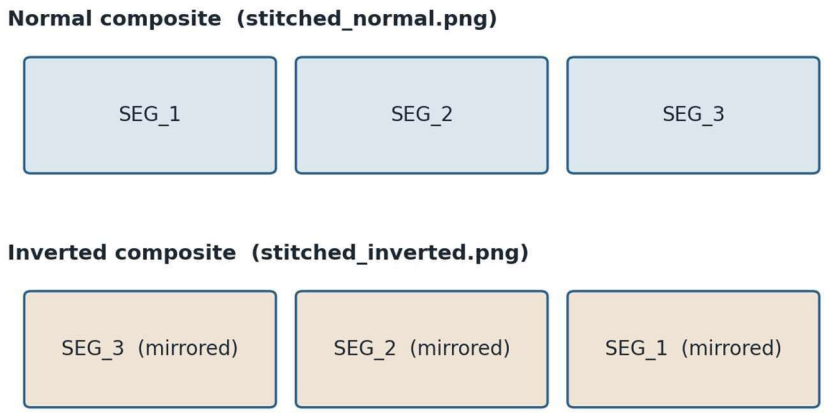


Figure 5. The inverted composite mirrors each segment individually and reverses their left-to-right order, which together mirror the entire composite.

6. Configuration and Tunable Parameters

Every stage in the pipeline is controlled by a small set of named, adjustable parameters — brightness thresholds, smoothing window sizes, search distances, and similar values — collected together at the top of the script rather than buried inside the processing logic. This makes the system easy to re-tune for a different imaging instrument, lighting condition, or segment layout without needing to modify the underlying algorithms themselves. The column-detection stages (Stage 1 and Stage 2) carry the most tunable parameters, reflecting that boundary detection from brightness profiles is the most sensitive part of the pipeline.

Figure 3. Tunable Configuration Parameters per Stage

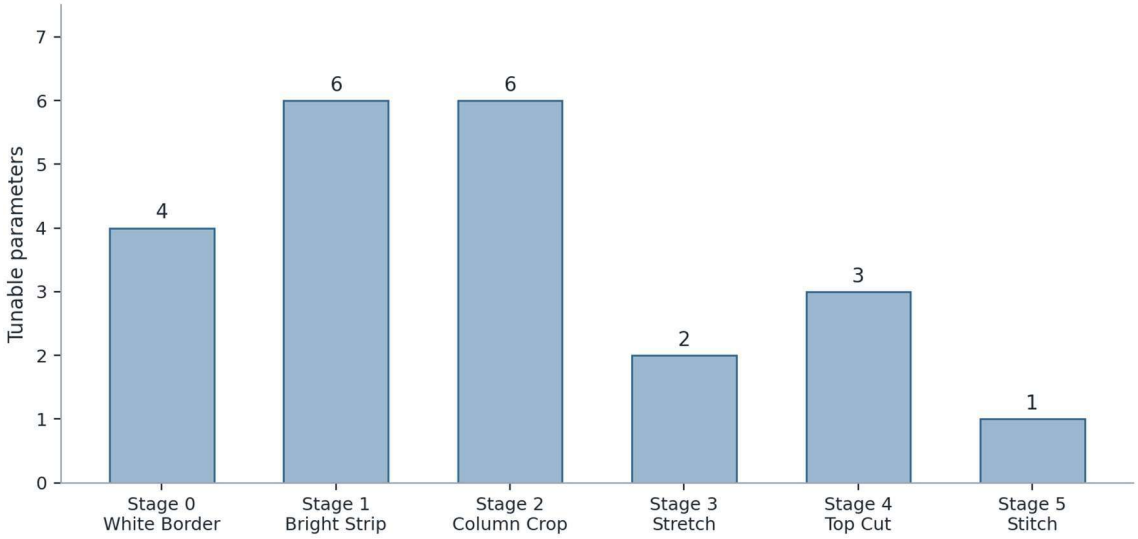


Figure 3. Number of adjustable configuration parameters exposed by each pipeline stage.

7. Performance Timing

The script includes a simple built-in timing utility that records how long each major phase of a run takes — parallel segment processing, saving the normal composite, and saving the inverted composite — and prints a short text-based bar report once the run finishes, alongside the total elapsed time. Because parallel segment processing involves the full five-stage correction pipeline run three times concurrently, it is expected to be the dominant share of total runtime, with the two stitching-and-save phases contributing comparatively little, as illustrated below.

Figure 4. Built-in Timing Report — Representative Profile
(illustrative; actual split varies with image size and hardware)

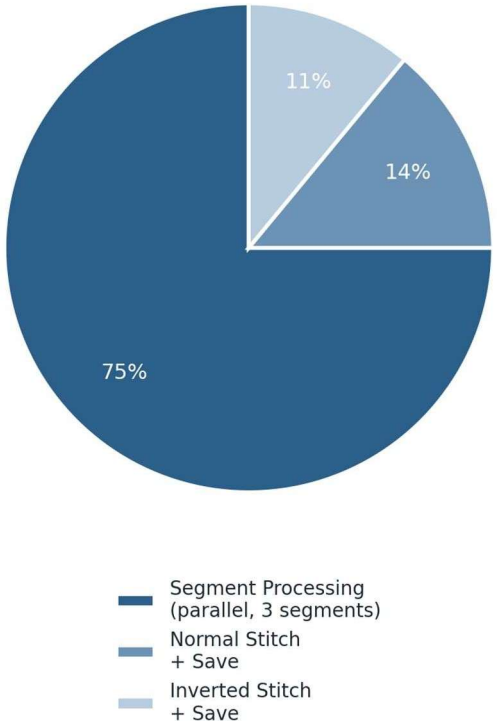
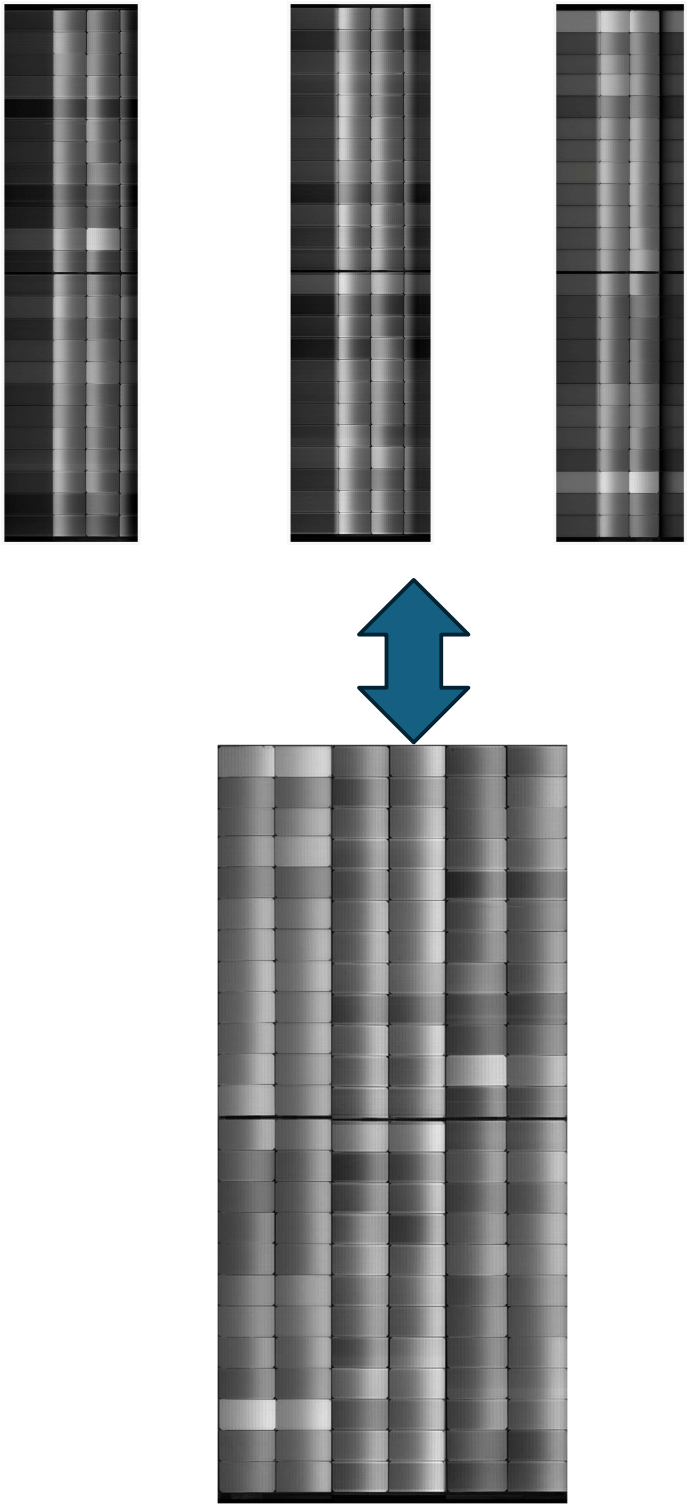


Figure 4. Representative share of total runtime across the three phases tracked by the built-in timer.

8. Output Organization

Results are written to an output folder named after the input batch, inside a fixed subfolder reserved for stitched results. Each run produces exactly two image files there: stitched_normal.png, the corrected and aligned left-to-right composite, and stitched_inverted.png, its fully mirrored counterpart. No intermediate per-segment files are kept by default, since every correction stage operates entirely in memory and only the two final composites are written to disk.



9. Conclusion

This project demonstrates a complete pipeline for turning three raw, independently captured photographs into a single, well-aligned panoramic composite. By combining brightness-profile analysis, gradient-based border detection, and instrument-specific geometric correction, the system is able to automatically locate and isolate the meaningful content of each segment without manual cropping, while parallel execution keeps processing time close to that of a single segment rather than three. Producing both a normal and a fully mirrored composite from the same corrected segments adds flexibility for downstream review without any extra processing cost.

Automated Multi-Panel Image Processing Pipeline

Fisheye Correction, Adaptive Grid Detection, and Cell-Wise Image Stitching

1. Introduction

Thermal imaging cameras are increasingly used in industrial inspection, surveillance, robotics, and research applications. Like any optical system, thermal cameras with wide field-of-view lenses introduce fisheye-type lens distortion, which must be corrected before the captured imagery can be used for accurate spatial measurement or analysis. This document describes the methodology, challenges, and solutions developed while building a distortion correction pipeline for a low-cost thermal camera (Seek Thermal sensor), along with a custom data-capture GUI built to support the work.

2. Problem Statement

Standard camera calibration relies on detecting a printed black-and-white checkerboard pattern across several images captured from different viewing angles. Each detected internal corner provides a known correspondence used to estimate the camera's intrinsic matrix (K) and distortion coefficients (D).

This approach could not be applied directly to the thermal camera. A thermal sensor measures emitted infrared radiation rather than visible reflectance, so a printed black-and-white checkerboard appears as a single, near-uniform-temperature surface with no visible pattern, making standard corner detection impossible.

3. Approach 1: Metal-Strip Checkerboard Pattern

3.1 Methodology

Following the method described in a 2018 study, thermal contrast was introduced into the checkerboard by attaching thin metal strips over every alternate square (covering either all the white or all the black squares of a standard pattern). Metal differs from paper in both emissivity and reflectivity, so it produces a measurable thermal contrast between the metal-covered and uncovered squares that is visible to the thermal sensor. Calibration images of this modified checkerboard were captured from multiple viewing angles, as required for intrinsic camera calibration.

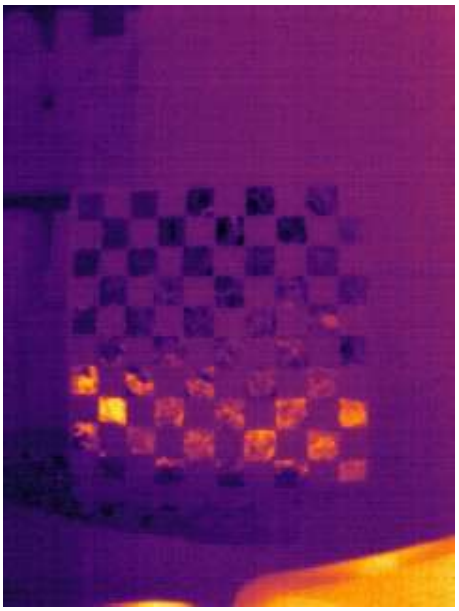


Figure 1: Thermal image of the metal-strip checkerboard pattern.

3.2 Limitations

Although the metal-strip pattern was distinguishable, the captured images were noisy due to ambient thermal interference — heat signatures and reflections from surrounding objects and walls bled into the pattern. This inconsistency caused the corner-detection algorithm to detect a different number and arrangement of internal corners across images of the same physical board, for example a 5×7 grid in one frame and a 6×5 grid in another. Since calibration requires a fixed, consistent grid size across all images, this approach could not be used directly.

4. Approach 2: High-Intensity Illumination

A more reliable solution was found in a 2011 IEEE paper by Saeed Yahyanejad, Jakub Misiorny, and Bernhard Rinner, which proposes illuminating a standard checkerboard with a high-intensity light source. Black squares absorb a larger proportion of the incident radiation while white squares reflect more of it, producing clear thermal contrast between the two without any physical modification of the checkerboard itself.

This method was implemented using a tungsten filament bulb as the high-intensity illumination source, positioned to evenly light the checkerboard. The resulting thermal frames showed a clean, repeatable checkerboard pattern and produced far more consistent corner detection than the metal-strip approach.

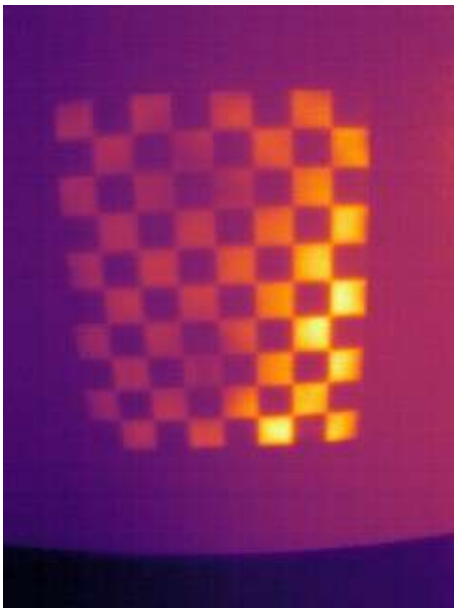


Figure 2: Thermal image of the checkerboard under tungsten-bulb illumination.

5. Image Preprocessing for Robust Corner Detection

5.1 CLAHE

Even with improved thermal contrast, raw captured frames were not consistently processed by the corner-detection algorithm due to residual noise and uneven illumination. Contrast Limited Adaptive Histogram Equalization (CLAHE) was applied to each of the 30 captured calibration images to locally enhance contrast before running corner detection.

5.2 Otsu Thresholding

CLAHE alone improved detection on some frames but failed on others. As a complementary technique, Otsu's thresholding was applied to the same 30 raw images to binarize the checkerboard pattern. A number of images that failed to yield a complete, consistent corner set under CLAHE succeeded under Otsu, and vice versa — the two techniques compensated for each other's weaknesses on different frames.

5.3 Combined Dataset and Final Selection

To maximize the number of usable calibration frames, both CLAHE-processed and Otsu-processed versions of all 30 source images were generated and passed through corner detection independently, producing a combined pool of 60 candidate frames. Out of this pool, 41 images yielded complete and consistent checkerboard-corner detections and were retained for camera calibration.

6. Camera Calibration

Using the 41 successfully detected calibration images, the camera's intrinsic parameters were

computed: the camera matrix (K), describing focal length and principal point, and the distortion coefficients (D), describing the lens's radial distortion.

Camera Matrix (K)

332.0018509	0.0000000	129.2125507
0.0000000	332.8387203	149.6005581
0.0000000	0.0000000	1.0000000

Distortion Coefficients (D)

-0.1118981	1.1949863	-10.6470181	25.4518617

7. Distortion Correction

7.1 Initial Issue: Image Dimension Mismatch

Applying the computed K and D values to undistort real (non-calibration) thermal images initially produced large black, invalid regions in the output rather than a clean corrected image.

7.2 Resolution

Further investigation showed that the resolution (width × height) of the images used during calibration must match, or be properly scaled to, the resolution of the images being corrected, since K and D are defined relative to a specific image size. After capturing and testing images at matching dimensions, and applying dimension scaling where required, the undistortion produced satisfactory and visually consistent results across the test set.

8. Software Development: Capture-Enabled Thermal GUI

8.1 Limitation of the Existing Seek Thermal Software

The Seek Thermal vendor software, originally used to operate the camera, did not provide a built-in option to save still screenshots of the live thermal feed — a capability needed to collect both calibration and test images.

8.2 Enhancements and GUI Features

The underlying capture script was modified to bind the 'S' key to saving a screenshot of the current frame. This was extended into a complete graphical interface, shown in Figure 3, offering Capture (single-frame screenshot), Record (continuous video capture), Rotate (clockwise / anticlockwise frame rotation), and Quit — improving the overall usability of the data-collection workflow.

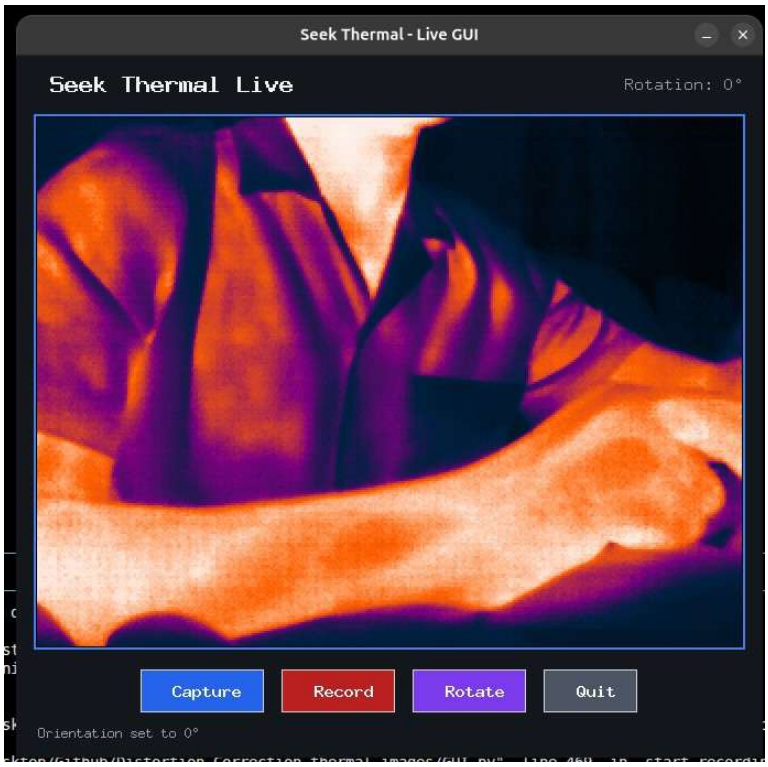


Figure 3: Custom-built Seek Thermal Live GUI with capture, record, and rotate controls.

9. Conclusion

This work demonstrates an end-to-end pipeline for camera calibration and fisheye distortion correction on a low-cost thermal imaging sensor. The fundamental challenge of checkerboard invisibility in thermal imagery was overcome through high-intensity illumination, and calibration robustness was improved through combined CLAHE and Otsu preprocessing of the captured frames. The resulting calibration parameters, the corrected output, and the custom data-collection GUI together form a complete toolkit for thermal lens-distortion correction.